

MEMORIA TSI PRÁCTICA 3

Ejercicio 1

La implementación de este ejercicio se ha diseñado priorizando la eficiencia del planificador. En el archivo de dominio, la decisión más relevante ha sido crear los tipos `tipoPersonaje` y `tipoRecurso`. Esto permite usar constantes genéricas (como `Hobbit` o `Mithril`) y vincularlas a los objetos físicos del mapa mediante los predicados `personajeEs`, `recursoEs` y `puedeExtraer`.

Otra decisión importante ha sido la creación del predicado (`recursoExtraido ?tipoRec`). Inicialmente, utilicé un cuantificador existencial (`exists`) en el `:goal` para comprobar si algún personaje estaba trabajando en el recurso. Sin embargo, *Fast Downward* compila las condiciones universales/existenciales en el objetivo transformándolas en axiomas. Dado que el alias exigido (`seq-opt-lmcut`) es un algoritmo de búsqueda óptima que no soporta axiomas, esta idea fallaba. La solución adoptada es que la acción `ExtraerRecurso` dispare el predicado (`recursoExtraido ?tipoRec`), simplificando la evaluación del estado final.

Por último, en el archivo de problema he optado por instanciar únicamente la información estrictamente necesaria (sólo los nodos de `Mithril` y `Alimento`). Esto minimiza el factor de ramificación y la cantidad de memoria consumida, evitando proporcionarle al algoritmo información irrelevante que ampliaría innecesariamente el espacio de búsqueda. Asimismo, he hecho uso de la Hipótesis del Mundo Cerrado para el estado de "Enano2", pues al no declararlo explícitamente como `disponible` en el estado inicial, el planificador lo asume como falso.

Análisis del fichero `output.sas`:

El archivo `output.sas` contiene el problema PDDL traducido a una representación SAS+ finita y completamente instanciada (*grounded*), que es la que el planificador utiliza para la búsqueda. Al analizarlo, se identifican las siguientes secciones clave:

- **Variables:** Delimitadas por `begin_variable` y `end_variable`. El traductor agrupa predicados mutuamente excluyentes en variables multivalor. Por ejemplo, en lugar de manejar múltiples hechos booleanos para la ubicación de "hobbit1", crea una única variable (`var3`) cuyos valores posibles representan cada una de las 20 ciudades. Para estados binarios (como (`disponible enano1`)), crea variables que pueden valer 0 o 1.
- **Estado inicial y objetivo:** Bloques (`begin_state`, `begin_goal`) que definen el punto de partida y de llegada del problema utilizando los índices de las variables previamente definidas.
- **Operadores:** Delimitados por `begin_operator` y `end_operator`. Representan las acciones instanciadas. Ya no existen variables lógicas (`?per`, `?loc`), sino acciones concretas (ej. `viajar enano1 edoras helmsdeep`).

Al revisar los operadores correspondientes a la acción `extraerrecurso`, se observa que solo hay dos instancias en todo el problema:

1. `extraerrecurso enano1 moria mithril`
2. `extraerrecurso hobbit1 hobbiton alimento`

Esta limitación de instancias se debe a las restricciones definidas. Estas dos son las únicas combinaciones posibles dadas las posiciones de los recursos y los permisos de extracción de cada personaje. Cabe destacar que el personaje "enano2" no posee ninguna instancia de este operador, pues comenzó el problema sin estar disponible y, al no existir ninguna acción capaz de alterar ese estado, el planificador lo ha descartado directamente, optimizando así la búsqueda.

Uso de IA Generativa:

Para el desarrollo de este ejercicio se ha utilizado el modelo *Gemini* como asistente de programación en las siguientes tareas:

1. **Generación de código repetitivo:** Solicité la transcripción del grafo de conexiones del mapa a formato PDDL para la sección `:init` del problema. Aunque requirió supervisión y corrección de algunos enlaces erróneos, agilizó significativamente la escritura.
2. **Resolución de errores de ejecución:** Ante el error devuelto por *Fast Downward* relativo a la falta de soporte para axiomas ("*This configuration does not support axioms*"), revisé los errores frecuentes de las diapositivas y consulté a Gemini. En las diapositivas identifiqué que el uso de cuantificadores existenciales (`exists`) en el bloque `:goal` forzaba la creación de predicados derivados incompatibles con la heurística LM-cut, y Gemini me recomendó el uso del predicado `(recursoExtraido ?tipoRec)`.
3. **Comprensión del archivo `output.sas`:** Lo utilicé para obtener una primera explicación de la estructura interna del fichero `output.sas`, para después interpretar yo cada uno de los bloques.

Ejercicio 2

Para abordar la formación y el movimiento de la Comunidad, introduje los predicados (`comunidadFormada`) y (`enComunidad ?per`). Modifiqué la acción original `Viajar` añadiendo la precondition (`not (enComunidad ?per)`), garantizando que, una vez un personaje se une a la Comunidad, deja de poder moverse individualmente. También creé `formarComunidad` y `viajarComunidad`, que exigen que los integrantes (un Hobbit y un Mago) se desplacen juntos, evaluando y modificando sus posiciones de forma simultánea.

En cuanto a la recolección de objetos, se pedía unificarla en una única acción `recogerObjeto`. Para conseguirlo, creé los predicados (`esAnillo`) y (`esEquipamiento`). Así, puedo comprobar las restricciones necesarias según el tipo de objeto con el que se está tratando.

Para la materialización del chaleco de Mithril (`materializarChaleco`), utilizo un cuantificador existencial (`exists`) para comprobar si algún personaje está trabajando en la extracción de Mithril en esa misma ubicación, sin importar qué personaje concreto sea.

Finalmente, la ubicación de destrucción del anillo no la fijé estáticamente en el dominio. En su lugar, eso el predicado (`lugarDestruccion ?loc`), instanciándolo a `Orodruin` en el fichero del problema. Esto hace que el dominio sea genérico y reutilizable si el destino cambiase en un futuro.

Ejercicio 3

En este caso, el código base es el del ejercicio anterior, modificando y adaptando la lógica de las acciones para manejar una comunidad de mayor tamaño. Se han introducido dos grandes mejoras para hacer frente a la complejidad del problema:

- **Ruptura de simetrías:** Se ha creado el predicado (`menor ?per1 - Personaje ?per2 - Personaje`). Se usa en los apartados b, c y d porque tenemos más de un Hobbit en la comunidad y, sin añadir esta precondition sobre su orden, el planificador comprobaba muchas permutaciones idénticas, perdiendo mucho tiempo.
- **La Comunidad como "vehículo":** Para evitar el colapso del planificador en el último apartado, se ha rediseñado la lógica de movimiento mediante el predicado (`comunidadEn ?loc`). Al formar la comunidad, los personajes "desaparecen" del mapa individual y pasan a formar parte de un único ente. Así, la acción `viajarComunidad` pasa de requerir hasta 7 parámetros (los personajes individuales más origen y destino) a necesitar únicamente 2 (origen y destino). Aún así, se han mantenido todos los parámetros en la acción, pues así se indica en el guión. Sin embargo, al no tener que hacer las comprobaciones de la ubicación de cada personaje, se ahorra bastante tiempo.

Tamaño de la Comunidad	Tiempo de Planificación ("Planner time")	Longitud del Plan encontrado ("Plan length")
(a): 1 hobbit y 1 mago	0.25 s	14
(b): 2 hobbits y 1 mago	0.33 s	16
(c): 3 hobbits y 1 mago	0.35 s	17
(d): 3 hobbits, 1 mago y 1 elfo	698.07 s	23

Discusión sobre la Complejidad Computacional

A la vista de los resultados, el tiempo de planificación crece de forma exponencial a medida que aumenta el tamaño de la Comunidad. Este incremento brutal de los tiempos se explica por la fase de instanciación. Todo proceso de planificación es un problema de búsqueda donde las variables se convierten en objetos instanciados. La cantidad de acciones instanciadas crece combinatoriamente en función del número de objetos del problema y del número de parámetros de las acciones: si tenemos una acción con p parámetros y O objetos posibles para cada uno, se tienen O^p posibles acciones a instanciar. Al añadir más personajes a las acciones `viajarComunidad` y `formarComunidad`, estamos incrementando p , lo que provoca que el espacio de estados explote.

El salto drástico de 0.35s en el caso c a 698s en el caso d ilustra perfectamente la explosión combinatoria del problema. Al requerir a un quinto miembro (el Elfo) situado en el extremo opuesto del mapa, el planificador se ve forzado a explorar el entrelazado de los movimientos individuales de todos los personajes a través del mapa para calcular el punto de reunión óptimo (ya que estamos forzando una búsqueda óptima con el algoritmo A*). A esto se suman los movimientos del resto de personajes disponibles, pero innecesarios para formar la comunidad, que el algoritmo también debe evaluar y descartar.

¿Cómo escala la resolución de los problemas a medida que incorporamos más acciones, objetos y/o nodos del mapa?

- **Más objetos o nodos:** Afectan a la fase inicial de instanciación. El planificador debe generar combinaciones de variables y objetos, creciendo el espacio de forma combinatoria (O^p). Un nodo extra puede crear miles de combinaciones nuevas. Por ejemplo, al haber más objetos (personajes) que no forman parte de la comunidad, estos se quedan "disponibles" y el planificador empieza a generar combinaciones de ellos viajando por todo el mapa.
- **Más acciones:** Incrementan el factor de ramificación en el árbol de búsqueda. Al haber más operadores posibles en cada estado, el algoritmo heurístico tiene que evaluar muchísimos más caminos, lo que dispara el tiempo de resolución.

¿El planificador proporcionaría con rapidez un plan si nuestra Comunidad estuviese formada por 4 hobbits, 1 mago, 1 elfo, 1 enano y 2 humanos?

No, dado que la complejidad crece exponencialmente, encontrar una solución para un problema con 9 personajes es casi imposible. Ya en el caso d (5 personajes) le cuesta mucho (11 minutos).

¿Es posible que, al igual en la Práctica 2, surjan soluciones simétricas? ¿De qué modo podrían aparecer estas y qué impacto tendrían en el rendimiento?

Sí, surgen principalmente de dos modos:

- **Simetría de permutación de parámetros:** Para el planificador, evaluar una comunidad con (Hobbit1, Hobbit4, Mago1) es un nodo distinto a evaluarla con (Hobbit4, Hobbit1, Mago1). Son los mismos objetos, pero en variables diferentes.
- **Simetría de identidad de objetos:** En el estado inicial existen personajes que son idénticos para el planificador. Por ejemplo, los personajes Hobbit 1, 2 y 3 comienzan todos en la misma localización (Hobbiton) y tienen las mismas capacidades. Para el algoritmo, coger al 1 y al 2 es una rama distinta a coger al 1 y al 3. Al ser tratados como objetos diferenciados, el planificador explora múltiples ramas que semánticamente representan la misma realidad.

Para solucionar ambas problemáticas, se añadió el predicado **menor** que establece un orden estricto entre los personajes, limitando el árbol a una única combinación válida. Este cambio, junto con el de considerar la comunidad como un único objeto, supuso un gran impacto en el rendimiento, pues inicialmente al caso d no acababa nunca (ahora tarda algo

más de 11 minutos, pero acaba) y el apartado c tardaba en encontrar solución casi 20 minutos (ahora necesita menos de un minuto).

Ejercicio 4

En este ejercicio, incluyo nuevos tipos de personaje (*Orco*, *Humano*, *Corsario*) y edificios (*TorreHechiceria*, *Extractor*), junto con las acciones *Construir*, *CrearUrukHai* y *RecogerEspecia*.

Para modelar la restricción de que los Enanos se niegan a extraer materiales que van a ser usados con fines malignos, en lugar de añadir restricciones en la acción *ExtraerRecurso*, que era complicado pues no sabía esos recursos para qué iban a ser usados después, opté por añadir las restricciones en la acción *Construir*. Mediante la anidación de un *forall* (para iterar sobre los recursos que necesita el edificio) y un *exists* (para verificar quién lo está extrayendo), añadí la condición (*not (personajeEs ?trabajador Enano)*). De este modo, si el planificador intenta validar la construcción de la Torre o del Extractor basándose en el trabajo de un Enano, la precondition falla, forzando al algoritmo a usar otros personajes.

Otra decisión fue la definición del lugar de construcción de la Torre de Hechicería. En lugar de forzar en el dominio que la torre deba construirse en Isengard, diseñé la acción *Construir* de forma genérica, y la restricción de la ubicación la añadí al fichero de problema, poniendo (*edificioConstruido TorreHechiceria Isengard*) en el bloque *:goal*. Esto mantiene el dominio reutilizable y modular.

Uso de IA Generativa

Al ejecutar el planificador con el algoritmo pedido (*seq-sat-lama-2011*) y el límite de 200 segundos, la ejecución se agotaba sin encontrar ningún plan. Pedí a Gemini estrategias de optimización y detectó dos cuellos de botella:

1. **Explosión combinatoria en cuantificadores:** Originalmente, en la acción *Construir*, el *exists* buscaba a un trabajador y su localización: (*?trabajador - Personaje ?locRec - Localizacion*). Al tener 16 personajes y 20 localizaciones, la instanciación de este axioma creaba un espacio de estados enorme. La IA sugirió crear un predicado auxiliar (*extrayendoRecurso ?per ?rec*), pues no nos importa la localización donde se esté extrayendo el recurso, solo el hecho de que esté siendo extraído. Al disparar este efecto en *ExtraerRecurso*, pude eliminar la variable de localización *?locRec* del cuantificador existencial, reduciendo exponencialmente las combinaciones a evaluar por el planificador.
2. **Ramas redundantes:** El planificador perdía tiempo asignando múltiples trabajadores al mismo recurso (ej. tres orcos distintos extrayendo el mismo nodo de madera). La IA sugirió añadir la precondition (*not (recursoExtraido ?tipoRec)*) en la acción *ExtraerRecurso*.

Gracias a esta reestructuración guiada, el planificador fue capaz no solo de resolver el problema dentro de los 200 segundos, sino de encontrar rápidamente una primera solución y optimizarla hasta generar tres planes válidos (con costes decrecientes de 51, 49 y 48).

Ejercicio 5

Para este último apartado se toma como base el código del Ejercicio 1, integrando la capacidad de trabajar con variables numéricas. En el fichero de dominio, tuve que definir un nuevo bloque (`:functions`). Dentro de este, declaré (`total-cost`), para acumular el coste global de la ejecución, y (`distancia ?origen ?destino`), para registrar el coste de viajar entre dos localizaciones. La única acción modificada fue `Viajar`, añadiendo (`increase (total-cost) (distancia ?origen ?destino)`), para que cada desplazamiento incremente el contador global en función de la ruta tomada.

Sobre la instanciación de las distancias, a diferencia de los predicados lógicos, donde se aplica la Hipótesis del Mundo Cerrado para asumir como falsos aquellos no declarados explícitamente, los valores numéricos no asumen ningún valor por defecto. Por ello, he tenido que inicializar el (`total-cost`) a 0 y declarar todos los costes de los caminos, incluyendo aquellos que tienen un coste unitario.

Por último, añadí el bloque (`:metric minimize (total-cost)`) al final del fichero de problema. Con esta instrucción, el planificador deja de buscar el plan más corto en número de acciones y pasa a buscar el plan que minimice el valor acumulado en dicha función.

Comparación de resultados con el Ejercicio 1:

En el Ejercicio 1, al no haber costes definidos, el planificador busca el plan con el menor número de acciones (coste unitario). En el Ejercicio 5, al usar la variable `total-cost`, busca la ruta más barata, aunque eso implique dar más pasos. La diferencia en las soluciones está en el camino que toma `Hobbit1`.

- **En el Ejercicio 1:** Lothlorien -> AmonHen -> Fangorn -> Isengard -> HelmsDeep -> Tharbad -> Hobbiton. Al carecer de costes, el algoritmo busca la solución con el menor número de acciones posibles.
- **En el Ejercicio 5:** Lothlorien -> AmonHen -> DeadMarshes -> MinasMorgul -> MinasTirith -> Edoras -> HelmsDeep -> Tharbad -> Hobbiton. Toma 8 acciones de viaje, dando un rodeo que antes no daba porque el camino entre `Fangorn` y `AmonHen` ahora tiene un coste de 8.

En conclusión, al introducir los costes en el camino vemos como un plan con más acciones puede ser numéricamente más óptimo que uno más corto en pasos.